

The Stretto Execution Engine for LLM-Augmented Data Systems

Objective, Problem, Pipeline, and Method

One-Sentence Summary

Core idea

How do we execute database-style queries over text, tables, images, and other unstructured data using LLMs — while controlling both quality and cost/runtime?

Instead of using the same expensive LLM everywhere, Stretto tries to choose the cheapest execution plan that still satisfies global quality constraints such as precision and recall.

**Minimize runtime/cost
subject to query-level quality guarantees.**

Why Is This Problem New?

Classical databases execute operators such as:

- filters,
- joins,
- projections,
- aggregations.

These operators are usually deterministic and cheap.

LLM-augmented data systems add semantic operators:

- “Does this report mention diabetes?”
- “Extract the diagnosis from this PDF.”
- “Does this image show an abnormal lung scan?”

These operators are expensive, stochastic, and have variable accuracy.

Simple Example

Suppose we have clinical reports and a patient table.

```
SELECT patient_id
FROM reports JOIN patients USING(patient_id)
WHERE semantic_filter(report, "mentions diabetes")
      AND age > 60;
```

The classical predicate `age > 60` is cheap and exact.

The semantic predicate `mentions diabetes` may be implemented by:

- keyword search,
- embedding similarity,
- small LLM,
- large LLM,
- cascade of several models.

Stretto decides which implementation to use and in what order.

Main Objective

Input

A logical query plan containing both classical relational operators and semantic operators.

User constraints

Global quality targets, for example:

$$\text{precision} \geq 0.90, \quad \text{recall} \geq 0.85$$

Output

A physical execution plan that chooses concrete operator implementations, thresholds, model sizes, KV-cache variants, and ordering.

The optimization goal is:

$$\min_{\text{plan}} \text{runtime}(\text{plan}) \quad \text{s.t.} \quad \text{quality}(\text{plan}) \geq \text{target}.$$

The Main Problem

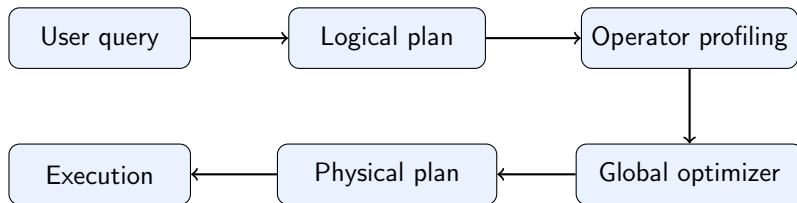
The hard part is not only choosing a good model for one operator.

The real problem is **global query planning**:

- semantic operators appear in pipelines;
- errors propagate across the pipeline;
- cheap early operators may remove tuples needed later;
- expensive later operators may compensate for earlier uncertainty;
- per-operator guarantees do not imply a good end-to-end guarantee.

Stretto optimizes the full pipeline, not each operator separately.

Pipeline of the Solution



- 1 Translate user query into a logical plan.
- 2 Identify semantic operators.
- 3 Profile candidate implementations on a sample.
- 4 Optimize the plan under global quality constraints.
- 5 Execute the chosen physical plan.

Logical Plan vs Physical Plan

Logical plan

Specifies **what** must be computed.

Example:

- scan reports;
- filter reports mentioning diabetes;
- join with patient table;
- return patient IDs.

Physical plan

Specifies **how** to compute it.

Example:

- first apply BM25;
- then embedding threshold;
- then small LLM;
- verify unsure cases with large LLM.

Stretto searches over physical implementations for each semantic operator.

Semantic Operators

Stretto assumes queries may contain operators whose semantics require language, vision, or multimodal understanding.

Examples:

Operator	Example
Semantic filter	keep reports mentioning diabetes
Semantic map	extract diagnosis from report
Semantic join	match product with relevant review
Semantic aggregation	summarize all complaints by category

Each semantic operator can have many implementations with different runtime–accuracy behavior.

Physical Operator Alternatives

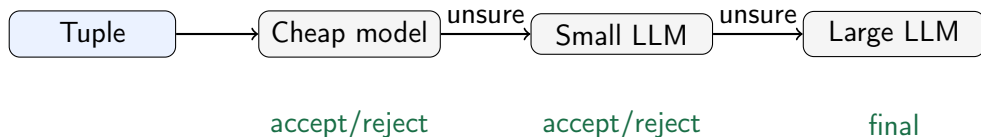
For one semantic filter, Stretto may consider several implementations.

Implementation	Cost	Quality
Keyword/BM25	very low	brittle
Embedding threshold	low	moderate
Small LLM	medium	good
Large LLM	high	strong
Cascade	adaptive	tunable

A cascade is often best in practice:

cheap filter → small model → large model only for unsure cases.

Operator Cascades



The intuition is simple: most easy tuples should be handled cheaply. Only ambiguous tuples should reach expensive models.

Profiling Step

Before optimization, Stretto profiles candidate operators on a sample of data.

For each candidate implementation, it estimates:

- runtime;
- selectivity;
- output scores or decisions;
- precision and recall behavior;
- sensitivity to thresholds.

This profiling allows the optimizer to simulate many execution plans without repeatedly calling expensive LLMs.

Stretto formulates planning as constrained optimization.

Decision variables include:

- which model to use;
- which KV-cache compression level to use;
- which thresholds to apply;
- whether to use a cascade;
- how to order semantic operators.

The optimization problem is mixed discrete/continuous, so Stretto relaxes discrete choices into differentiable soft choices and uses gradient-based search.

Why Bayesian Quality Estimation?

Quality estimates come from finite samples, so they are uncertain.

Example:

$$\text{estimated recall} = 0.91$$

This does not mean the true recall is exactly 0.91.

Stretto uses Bayesian credible intervals to reason about uncertainty:

$$P(\text{recall} \geq 0.85 \mid \text{sample}) \text{ is high.}$$

This prevents the optimizer from selecting plans that only look good due to sampling noise.

KV Cache as a Physical Design Choice

Transformers store key/value states for previously processed tokens.

Normally, KV cache is used during generation to avoid recomputing previous tokens.

Stretto treats KV cache as a reusable representation of database items:

- precompute KV caches offline for documents/images;
- reuse them during query execution;
- compress them at different ratios;
- expose each compression level as a different physical operator.

More compression means faster execution but potentially lower accuracy.

Why KV Cache Matters

Without KV-cache variants, the search space is coarse:

small LLM or large LLM

With compressed KV-cache variants, Stretto gets a smoother runtime–quality spectrum:

small model $\rightarrow$ compressed cache $\rightarrow$ less compressed cache $\rightarrow$ large model

This makes gradient-based optimization more useful because the optimizer can navigate fine-grained trade-offs instead of choosing only from a few discrete models.

Query Order Optimization

Stretto also reorders operators.

Bad plan:

LLM filter on all reports → age filter

Better plan:

age filter → LLM filter on remaining reports

Relational operators are cheap and exact, so they should usually reduce the data before expensive semantic operators run.

How Stretto Differs from Simpler RAG

A simple RAG pipeline usually does:

retrieve chunks $\rightarrow$ prompt LLM $\rightarrow$ answer

Stretto instead treats the system as a database execution engine:

- query is represented as a plan;
- semantic operators have multiple implementations;
- operator choices are optimized globally;
- quality constraints are explicit;
- cost and latency are part of planning.

Toy Example: Movie Reviews

Question:

Find three interesting movies from 1998 with reviews mentioning strong atmosphere.

Possible plan:

- 1 Structured filter: `year = 1998`.
- 2 Cheap retrieval: BM25 or embeddings for “strong atmosphere”.
- 3 Small LLM: classify candidate reviews.
- 4 Large LLM: verify ambiguous cases.
- 5 Return top three rows with citations.

Stretto's question is not “can an LLM answer this?” but “what is the cheapest reliable execution strategy?”

Failure Modes and Limitations

Stretto reduces cost and gives stronger guarantees, but it does not remove all risks.

- Quality estimates depend on the profiling sample.
- Bad semantic parsers can produce bad logical plans.
- Semantic operators can still hallucinate or misclassify.
- KV-cache engineering is complex and model-dependent.
- Global guarantees are statistical, not absolute correctness guarantees.

The method is strongest when there is enough validation/profiling data to estimate operator behavior reliably.

Relation to Heterogeneous Document QA

For a project over PDFs, tables, CSVs, and semi-structured data, Stretto suggests a useful architecture:

- represent the user question as a query plan;
- separate structured filters from semantic filters;
- profile retrieval, reranking, and LLM verification modules;
- use cascades instead of one large model call;
- optimize for end-to-end answer quality, not isolated retrieval scores.

This is especially relevant for experiments where context size, retrieval depth, and LLM calls affect both accuracy and latency.

Key Takeaways

- 1 Stretto is about **LLM-aware query execution**, not just prompting.
- 2 The main problem is the **runtime–accuracy trade-off** of semantic operators.
- 3 The objective is to minimize cost while satisfying **global precision/recall constraints**.
- 4 The method combines profiling, Bayesian quality estimation, gradient-based optimization, cascades, and KV-cache variants.
- 5 The core architectural lesson: treat LLM calls as database operators that must be planned, optimized, and evaluated.

Gabriele Sanmartino, Matthias Urban, Paolo Papotti, and Carsten Binnig.
The Stretto Execution Engine for LLM-Augmented Data Systems.
arXiv:2602.04430, 2026.